

ENTENDIENDO UN PROYECTO

DESARROLLO WEB EN ENTORNO SERVIDOR

Nota: No olvides que el código debe estar documentado y escrito en inglés. Usaremos la convención de triples comillas para documentar el código.

1. NUEVA HOMEPAGE

1. Nuestro servidor actualmente solo tiene la página por defecto de Django. Vamos a intentar entender cómo cambiarlo. Dentro de nuestra carpeta de proyecto crea un archivo `views.py` con un contenido como el siguiente:

```
from django.http import HttpResponse
```

```
def homepage(request):  
    return HttpResponse("Hello World!")
```

[AI Assistant] Fix aplicado: En el original faltaba cerrar un paréntesis en `return` (`HttpResponse(...)`) y las comillas eran tipográficas curvas “ ”.

2. Ahora vamos a definir una URL que nos dirija a esta función. Esto lo hacemos dentro del archivo `urls.py`:

```
from django.contrib import admin  
from django.urls import path  
from . import views
```

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', views.homepage)  
]
```

[AI Assistant] Fix aplicado: En el original faltaba cerrar la comilla en `path('` y faltaban las importaciones de `admin` y `path`.

3. Prueba a ver qué tal el resultado. ¿Funciona? La página debería ser bastante fea. Vamos a intentar solucionarlo. Vamos a crear una carpeta `templates` a la misma altura que el resto del código y dentro añadiremos un `home.html` con una página de acogida de tu elección.

4. Ahora tenemos que decirle a Django que puede buscar plantillas dentro de esa carpeta. Eso lo hacemos desde los ajustes de Django. Buscamos la variable `TEMPLATES` y añadimos las siguientes líneas:

Nota: Siempre que modifiquemos `settings.py` tendremos que reiniciar el servidor.

```
import os
```

```
TEMPLATES = [  
    {  
        'DIRS': [os.path.join(BASE_DIR, 'templates')],  
    }  
]
```

[AI Assistant] Fix aplicado: Se incluye `import os` al inicio para evitar que `os.path.join` cause `NameError` si `os` no estuviera importado.

5. Solo nos queda modificar nuestra función `homepage` para que muestre nuestro nuevo `html`:

```
from django.shortcuts import render
```

```
def homepage(request):  
    return render(request, 'home.html')
```

[AI Assistant] Fix aplicado: Se corrigieron las comillas tipográficas “`home.html`” por comillas válidas ‘`home.html`’.

6. Comprueba que puedas verlo, ¿todo funciona? Ahora podemos empezar a usar assets estáticos como imágenes y archivos `css` o `js`. Vamos a crear una carpeta junto a `templates` que se llame `static` y dentro de esta carpeta 3 carpetas nuevas llamadas `js`, `css`, y `images`. Añade en `settings.py` estas líneas:

```
STATICFILES_DIRS = [  
    os.path.join(BASE_DIR, 'static')  
]
```

7. ¿Cómo funciona esto? Dentro de nuestros HTML ahora podremos ejecutar código sencillo para importar automáticamente estos recursos. Por ejemplo:

```
<!DOCTYPE html>
{% load static %}
<html>
    <head>
        <meta charset="UTF-8" />
        <title>Home</title>
        <link rel="stylesheet" href="{% static 'css/style.css' %}">
    </head>
</html>
```

8. Prueba a añadir una o dos imágenes, una hoja de estilos y un archivo javascript. Comprueba que todo funciona antes de continuar.

2. APLICACIONES

Nota: Cada aplicación de Django va a corresponder, grosso modo, a un dominio principal de nuestro análisis DDD. ¡Recuerda ejecutar siempre todos los comandos desde la carpeta raíz del proyecto Django, no desde la carpeta del entorno virtual!

9. Para crear aplicaciones necesitamos apagar el servidor y ejecutar el siguiente comando. Vamos a empezar con el dominio más sencillo que se nos ocurra. Buenos ejemplos son: 'Escaparate', 'Homepage' o 'Carrusel de fotos'.

```
python manage.py startapp NOMBRE_DE_LA_APP
```

[AI Assistant] Fix aplicado: Se retiró el símbolo \$ de la consola para evitar confusiones de sintaxis en PowerShell/CMD.

10. Se nos debería crear una carpeta con una serie de archivos muy parecidos a los que vimos anteriormente. Vamos a tocar los ajustes de Django para que reconozca la nueva App.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'NOMBRE_DE_LA_APP',
]
```

11. Cada App debería tener su propio conjunto de plantillas html. Creamos dentro de la carpeta de la app dos carpetas nuevas tal que estas: templates y dentro de templates creamos NOMBRE_DE_LA_APP. El único propósito de esto es crear una estructura de carpetas estandarizadas y claramente reconocibles.

12. Añade dentro de esta carpeta un html con la estructura de lo que será la página.

13. Ya podemos editar el views.py de nuestra app:

```
from django.shortcuts import render

def NOMBRE_DE_LA_PAGINA(request):
    return render(request, 'NOMBRE_DE_LA_APP/NOMBRE_DEL_HTML.html')
```

14. De la misma forma que anteriormente, solo nos queda vincular las rutas. Esto se hace en dos etapas:

En **NOMBRE_DE_LA_APP/urls.py**:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.NOMBRE_DE_LA_PAGINA)
]
```

En el **urls.py** principal del proyecto:

```
from django.contrib import admin
from django.urls import path, include
from . import views
```

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', views.homepage),  
    path('NOMBRE_DE_LA_APP/', include('NOMBRE_DE_LA_APP.urls'))  
]
```

15. Comprueba que todo funciona. Intenta añadir enlaces o un menú para navegar de forma cómoda de una página a otra. ¿Funciona todo bien?